

Bug修复日志

团队：第67组 | 日期：2026-06-01

一、Bug基本情况整理表格

以下是将三份 Bug 修复日志内容按您指定格式整理后的表格：

Bug #	Bug 描述	问题现象	问题原因	修复方案	修复结果	最终方案来源
1	驳回完成后的备注无法稳定保留	订单在“发布者驳回完成”后，页面需要显示驳回理由；但该信息不落库，刷新页面或重新加载订单详情后备注会消失	早期实现只关注订单状态流转，没有把“发布者最新驳回备注”作为持久化字段纳入订单实体	在订单实体中增加 latestRequesterNote，补齐 JPA 映射与仓储转换逻辑，并在前端订单详情页增加驳回备注展示区域	发布者驳回完成时必须填写备注；刷新订单详情后备注仍然存在；前端页面能正确展示最近一次驳回说明	人工
2	订单完成时间无法持久化	订单确认后，需要记录关闭时间供历史订单展示和后续评价流程使用；若未持久化该时间，订单历史信息不完整	初始实现只更新了订单状态，没有把“完成时间”完整写入数据库实体与仓储映射	在订单实体中增加 completedAt，补齐仓储层保存与回填，在订单详情返回对象中暴露完成时间，在订单确认完成动作中同步写入关闭时间	发布者确认后订单状态变为 COMPLETED；重新查询订单详情时完成时间正确返回；历史订单信息比修复前完整	人工

Bug #	Bug 描述	问题现象	问题原因	修复方案	修复结果	最终方案来源
3	申请记录和订单时间线重启后丢失	订单模块需要展示“申请记录”和“状态时间线”；服务重启后这些数据会丢失，影响演示和业务追踪	AI 直出的业务模型更偏向领域对象表达，但没有完整落到 MySQL 表结构	新增 <code>DemandApplication</code> 实体和 <code>OrderTimelineRecord</code> 实体及对应仓储，在 <code>JpaOrderRepository</code> 中补齐双向转换，增加初始化演示数据	申请记录可以从数据库稳定读取；时间线在刷新和重启后仍然保留；演示页可稳定展示多个状态下的订单数据	人工
4	系统通知点击后未读红点不消失（消息通知模块）	用户进入消息页面，系统通知 Tab 显示未读红点数字；点击通知阅读后红点不减少，紫色未读高亮背景不消失；刷新或轮询后未读依然存在	系统通知列表项没有绑定点击事件，对比私信会话项有点击事件，导致前端从未调用后端标记已读接口	为系统通知项添加 <code>@click</code> 事件，新增 <code>handleNotificationClick</code> 函数调用 <code>PUT /messages/read/{id}</code> 并即时更新本地状态，将 CSS 中 <code>cursor</code> 改为 <code>pointer</code>	点击未读通知后紫色背景消失、Tab红点减1；连续点击多条红点递减直至消失；刷新后已读状态保持；点击已读通知不重复请求	AI 方案
5	消息页面初次加载时未读红点闪烁	进入消息页面时，红点先不显示，约0.2-0.5秒后	<code>conversations</code> 和 <code>systemNotifications</code> 初始化为空数组（返回0）， <code>onMounted</code> 中异步加载数据，返回后红点突然出现	增加 <code>pageReady</code> 状态标记，在数据加载完成前不显示红点，等数据就绪后一次性展示完整状态	首次进入红点无明显闪烁；弱网环境下一次性展示红点；已读全部消息后红点不出现；轮询	人工

Bug #	Bug 描述	问题现象	问题原因	修复方案	修复结果	最终方案来源
	闪烁 (消息通知模块)	突然“弹出”，有明显闪烁/跳动			刷新期间红点平滑更新不闪烁	
6	前端评价提交 API 路径与后端不匹配 (评价模块)	用户点击“提交评价”后浏览器报错 404 Not Found, 后端无请求日志, 评价功能完全不可用	前后端三处不匹配: 路径 (/reviews vs /reviews/create)、传参方式 (JSON Body vs Query Parameters)、字段名 (reviewerId vs userId 等)	修改后端 Controller, 新增 CreateReviewRequest DTO, 接口改为 @PostMapping 接收 @RequestBody	前端 POST /api/reviews 返回 201; 数据库正确写入; 被评价者平均评分正确更新; 重复评价及非订单参与方评价被正确拦截	AI 方案 (人工决策采用)
7	评分边界值 0 分未在前端校验 (评价模块)	用户不点击评分星级组件直接提交评价, 前端发送 rating: 0, 后端返回“评分必须在 1 到 5 之间”, 用户体验差	el-rate 组件默认值为 0, Element Plus 表单校验规则的 trigger: 'change' 在值为 0 时不触发	在 handleSubmit 函数中增加手动边界检查, 在调用 formRef.validate() 之前先检查 rating 是否为 0	评分为 0 时点击提交弹出友好提示“请选择评分 (1-5 星)”; 选择 1-5 星后正常提交; 不产生后端网络请求	人工 (AI 建议的双重防线过于冗余, 未完全采纳)
8	删除评价后用户平均分重新计算	管理员删除某用户的唯一一条评价后, 该用户信	JPA @Transactional 方法内 DELETE 和 SELECT 的执行顺序受持久化上下文影响, 导致 newCount=0 但 newAvg≠null 的矛盾状态	在 deleteReview 中 delete() 后立即 flush(); 增强 recalculateUserAverageScore 防御性判断, newCount 为 0 时直接置为 0.0	删除唯一评价后平均分正确显示 0.0; 前端信用页不再出现 NaN; 有多条评价时删除一条后平均分	AI 方案

Bug #	Bug 描述	问题现象	问题原因	修复方案	修复结果	最终方案来源
	为 null (评价模块)	用页显示 NaN 或 null, 前端页面渲染异常			正确重新计算; 单元测试全部通过	

二、Bug修复详细记录

Bug 1：驳回完成后的备注无法稳定保留

问题现象

订单在“发布者驳回完成”后，页面需要显示驳回理由；但如果该信息不落库，刷新页面或重新加载订单详情后，备注会消失，无法支撑后续沟通与演示。

根因分析

早期实现只关注订单状态流转，没有把“发布者最新驳回备注”作为持久化字段纳入订单实体，导致该信息只存在于运行时对象中。

修复方案

- 在订单实体中增加 latestRequesterNote
- 在 JPA 映射与订单仓储转换逻辑中补齐该字段
- 在订单详情视图对象中返回该字段
- 在前端订单详情页中增加驳回备注展示区域

验证结果

- 发布者驳回完成时必须填写备注
- 刷新订单详情后备注仍然存在
- 前端页面能正确展示最近一次驳回说明

Bug 2：订单完成时间无法持久化

问题现象

订单确认完成后，需要记录关闭时间，供历史订单展示和后续评价流程使用；若未持久化该时间，订单历史信息不完整。

根因分析

初始实现只更新了订单状态，没有把“完成时间”完整写入数据库实体与仓储映射，导致状态变为 **COMPLETED** 后仍缺少完成时间。

修复方案

- 在订单实体中增加 **completedAt**
- 在仓储层补齐 **completedAt** 的保存与回填
- 在订单详情返回对象中暴露完成时间
- 在订单确认完成动作中同步写入关闭时间

验证结果

- 发布者确认完成后，订单状态变为 **COMPLETED**
- 重新查询订单详情时，完成时间能够正确返回
- 历史订单信息比修复前完整

Bug 3：申请记录和订单时间线重启后丢失

问题现象

订单模块需要展示“申请记录”和“状态时间线”；如果只在内存模型中维护，服务重启后这些数据会丢失，影响演示和业务追踪。

根因分析

AI 直出的业务模型更偏向领域对象表达，但没有完整落到 MySQL 表结构，导致申请记录和时间线缺少持久化支持。

修复方案

- 新增 **DemandApplication** 实体和对应仓储
- 新增 **OrderTimelineRecord** 实体和对应仓储
- 在 **JpaOrderRepository** 中补齐领域对象与数据库实体之间的双向转换
- 增加初始化演示数据，覆盖开放中、进行中、待确认、已完成等状态

验证结果

- 申请记录可以从数据库稳定读取
- 时间线在刷新和重启后仍然保留
- 演示页可稳定展示多个状态下的订单数据

Bug 4：系统通知点击后未读红点不消失

问题现象

1. 用户进入消息页面，系统通知 Tab 上显示未读红点数字
2. 用户点击系统通知列表中的某条通知，阅读其内容
3. 但 Tab 上的**未读红点数字没有减少**，通知项的紫色未读高亮背景也没有消失

4. 即使刷新页面或等待 10 秒轮询刷新，未读状态依然存在

根因分析

涉及文件：frontend/src/components/MessagesPage.vue

问题在于系统通知列表项没有绑定点击事件，而私信会话项是有点击事件的。

```
<!-- 系统通知项：缺少 @click 事件 -->
<div v-for="notif in systemNotifications" :key="notif.id"
      class="notification-item" :class="{ unread: !notif.read }">
  <div class="notif-content">{{ notif.content }}</div>
  <div class="notif-time">{{ formatTime(notif.createdAt) }}</div>
</div>
```

关键原因：

- 系统通知项（notification-item）没有绑定 @click 事件
- 对比私信会话项（conversation-item），它有 @click="selectConversation(conv)"，而 selectConversation 函数内部会调用 PUT /messages/read-conversation 接口标记已读
- 系统通知被点击后，前端从未调用后端 PUT /messages/read/{messageId} 接口来标记该通知为已读
- 后端数据库中 is_read 字段始终为 false，所以每次拉取通知列表，未读计数都不会减少

修复方案

涉及文件：frontend/src/components/MessagesPage.vue

步骤 1 —— 为系统通知项添加点击事件：

```
<div v-for="notif in systemNotifications" :key="notif.id"
      class="notification-item" :class="{ unread: !notif.read }"
      @click="handleNotificationClick(notif)">
  <div class="notif-content">{{ notif.content }}</div>
  <div class="notif-time">{{ formatTime(notif.createdAt) }}</div>
</div>
```

步骤 2 —— 新增 handleNotificationClick 函数：

```
const handleNotificationClick = async (notif) => {
  // 如果已经已读，不重复请求
  if (notif.read) return

  try {
    await axios.put(BASE + '/messages/read/' + notif.id, null, {
      params: { token: authStore.token }
    })
    // 本地状态即时更新，提升用户体验
    notif.read = true
  }
```

```
    } catch (e) {  
      showNotification('操作失败', '标记已读失败，请重试')  
    }  
  }  
}
```

步骤 3 —— 将 CSS 中 `notification-item` 的 `cursor: default` 改为 `cursor: pointer`，让用户知道通知是可以点击的。

验证结果

验证项	预期结果	实际结果
点击一条未读系统通知	该通知紫色背景消失，Tab 红点数字减 1	通过
连续点击多条未读通知	红点数字逐个递减直至消失	通过
刷新页面后再次查看	已读通知保持已读状态，红点不重现	通过
点击已读通知	不会重复发送 API 请求	通过

Bug 5：消息页面初次加载时未读红点闪烁

问题现象

- 1. 用户从首页点击“消息”进入消息页面
- 2. 页面瞬间渲染出私信和系统通知两个 Tab，此时红点不显示
- 3. 约 0.2~0.5 秒后（网络请求返回），红点数字突然“弹出”
- 4. 视觉效果上有一个明显的**闪烁/跳动**，体验不佳

根因分析

涉及文件： `frontend/src/components/MessagesPage.vue`

未读红点的显示依赖两个计算属性：

```
const chatUnreadCount = computed(() =>  
  conversations.value.reduce((sum, c) => sum + (c.unreadCount || 0), 0)  
)  
  
const systemUnreadCount = computed(() =>  
  systemNotifications.value.filter(n => !n.read).length  
)
```

问题链路：

- 1. 组件创建时，`conversations` 和 `systemNotifications` 初始化为空数组
- 2. 空数组导致计算属性返回 0，红点不显示
- 3. `onMounted` 中调用 `refreshData()`，发起**异步** HTTP 请求
- 4. 网络请求返回后，数据被赋值，红点突然出现——造成闪烁

```
// 初始值是空数组
const conversations = ref([])
const systemNotifications = ref([])

// 数据在 onMounted 中异步加载
onMounted(() => {
  if (!authStore.isLoggedIn) { router.push('/login'); return }
  refreshData() // 异步请求，不会阻塞页面渲染
  pollTimer = setInterval(refreshData, 10000)
})
```

修复方案

涉及文件：frontend/src/components/MessagesPage.vue

思路：增加一个 `pageReady` 状态标记，在数据加载完成前不显示红点，等数据就绪后一次性展示完整状态。

步骤 1 —— 新增状态变量：

```
const pageReady = ref(false)
```

步骤 2 —— 让 `refreshData` 变为 `async` 函数，在所有数据加载完成后标记就绪：

```
const refreshData = async () => {
  await fetchConversations()
  await fetchNotifications()
  pageReady.value = true
}
```

步骤 3 —— 模板中，红点仅在就绪后才显示：

```
<button :class="{ active: activeTab === 'chat' }" @click="activeTab = 'chat'">
  私信
  <span v-if="pageReady && chatUnreadCount > 0" class="tab-badge">
    {{ chatUnreadCount }}
  </span>
</button>

<button :class="{ active: activeTab === 'system' }" @click="activeTab = 'system'">
  系统通知
  <span v-if="pageReady && systemUnreadCount > 0" class="tab-badge">
    {{ systemUnreadCount }}
  </span>
</button>
```


验证结果

验证项	预期结果	实际结果
首次进入消息页面	红点不会先消失再出现，无明显闪烁	通过
模拟弱网环境（3G 网速）	页面等待数据加载完成后一次性展示红点	通过
已读完全部消息后进入页面	红点始终不出现，无闪动	通过
10 秒轮询刷新期间	红点数字平滑更新，不闪烁	通过

Bug 6：前端评价提交 API 路径与后端不匹配

问题现象

前端评价提交页面（`ReviewPage.vue`）在用户点击“提交评价”后，浏览器控制台报错 `404 Not Found`，后端无任何请求日志。评价功能完全无法使用。

复现步骤

1. 打开评价页面，切换到“提交评价” Tab
2. 填写订单 ID、评价人 ID、被评价人 ID、评分、评语
3. 点击“提交评价”按钮
4. 浏览器 Network 面板显示 `POST /api/reviews` 返回 404

根因分析

前端代码（`frontend/src/views/ReviewPage.vue:40`）：

```
const res = await api.post('/reviews', {
  orderId: Number(submitForm.orderId),
  reviewerId: Number(submitForm.reviewerId),
  revieweeId: Number(submitForm.revieweeId),
  rating: submitForm.rating,
  comment: submitForm.comment,
  reviewerRole: submitForm.reviewerRole,
})
```

后端代码（`backend/.../controller/ReviewController.java:32-33`）：

```
@PostMapping("/create") // ← 实际路径是 /reviews/create
public ResponseEntity<ApiResponse<Review>> createReview(
    @RequestParam Long orderId, // ← 期望 Query 参数
    @RequestParam Long userId, // ← 参数名是 userId, 不是 reviewerId
    @RequestParam Integer score, // ← 参数名是 score, 不是 rating
    @RequestParam(required = false) String content) {
```

三个不匹配点：

1. **路径不匹配**：前端请求 `/reviews`，后端监听 `/reviews/create`
2. **传参方式不匹配**：前端发 JSON Body，后端期望 Query Parameters (`@RequestParam`)
3. **字段名不匹配**：前端传 `reviewerId` / `revieweeId` / `rating`，后端期望 `userId` / `score` / `content`

根本原因是前后端由不同开发者并行开发，AI 辅助生成代码时未统一接口契约。

修复方案

采用方式：修改后端 Controller 接口，改为接收 JSON Body (`@RequestBody`)，与前端对齐。

修改文件：`backend/src/main/java/com/example/backend/controller/ReviewController.java`

修改内容：

```
// 新增请求 DTO（内联静态类）
public static class CreateReviewRequest {
    private Long orderId;
    private Long reviewerId;
    private Integer rating;    // 前端字段名
    private String comment;   // 前端字段名
    // getters & setters ...
}

// 修改接口
@PostMapping // ← 去掉 /create, 匹配前端 /reviews
public ResponseEntity<ApiResponse<Review>> createReview(
    @RequestBody CreateReviewRequest request) { // ← 接收 JSON Body

    Review review = reviewService.createReview(
        request.getOrderId(),
        request.getReviewerId(),
        request.getRating(),
        request.getComment()
    );
    // ...
}
```

说明：选择改动后端而非前端，因为后端改动范围可控（一个 Controller），且前端 `revieweeId` 本就应由后端根据订单自动推导（评价对方），不应由用户手动输入——这也避免了恶意评价他人的风险。

验证结果

- ☒ 前端 `POST /api/reviews` 返回 201 Created
- ☒ 数据库中正确写入评价记录
- ☒ 被评价者平均评分正确更新
- ☒ 重复评价被正确拦截（返回 400）
- ☒ 非订单参与方评价被正确拦截

经验教训

- 1. 前后端接口契约应提前以文档确认，不能仅依赖口头沟通或各自看设计文档
- 2. AI 生成的前后端代码需要交叉验证，让 AI 同时生成接口文档可以暴露不一致
- 3. 评价对象 (revieweeId) 应由后端根据订单推导，前端不应暴露此字段，既减少输入也提升安全性

修复方式对比

维度	纯人工调试	AI 辅助调试
定位耗时	~15 分钟（查看浏览器 Network → 搜索后端路由 → 对比参数）	~3 分钟（将报错信息和前后端代码片段提供给 AI）
定位准确度	准确	准确（AI 直接指出了三处不匹配）
修复方案	人工决定改后端	AI 提供了两种方案（改前端/改后端）并分析了各自的优缺点
最终采用	改后端（人工决策）	AI 建议改后端（理由：安全性更好，避免前端传入 revieweeId 可被篡改）

Bug 7：评分边界值 0 分未在前端校验

问题现象

用户不点击评分星级组件直接提交评价，前端发送 rating: 0 到后端，后端返回 "评分必须在 1 到 5 之间"。虽然数据未错误写入，但用户看到红色错误提示体验很差——前端应该在提交前就拦截。

复现步骤

- 1. 打开评价提交表单
- 2. 填写订单 ID、评价人 ID、被评价人 ID，但不点击评分星星
- 3. 直接点击“提交评价”
- 4. 等待后端响应后看到错误提示

根因分析

前端 el-rate 组件默认值为 0，而 Element Plus 的表单校验规则虽配置了 min: 1, max: 5，但 el-rate 组件在值为 0 时不触发 change 事件，导致校验规则的 trigger: 'change' 不生效。

```
<!-- ReviewPage.vue:177 -->
<el-rate v-model="submitForm.rating" :max="5" show-score />

<!-- 校验规则: trigger: 'change' 在 rating=0 时不触发 -->
rating: [
  { required: true, message: '请选择评分', trigger: 'change' },
  { type: 'number', min: 1, max: 5, message: '评分需在1-5之间', trigger: 'change' },
]
```

修复方案

在 `handleSubmit` 函数中增加手动边界检查，在调用 `formRef.validate()` 之前先检查 `rating` 是否为 0：

修改文件： `frontend/src/views/ReviewPage.vue`

```
async function handleSubmit() {
  // 新增：手动校验评分（el-rate 默认值为 0 时不触发 change 事件）
  if (submitForm.rating === 0) {
    ElMessage.warning('请选择评分（1-5 星）')
    return
  }

  if (!submitFormRef.value) return
  try {
    await submitFormRef.value.validate()
  } catch {
    return
  }
  // ... 后续提交逻辑
}
```

验证结果

- ☒ 评分为 0 时点击提交，弹出友好提示“请选择评分（1-5 星）”
- ☒ 选择 1-5 星后正常提交
- ☒ 不产生后端网络请求（前端提前拦截）

经验教训

1. Element Plus 的 `el-rate` 组件默认值为 0 时 `change` 事件不触发，这是组件库的已知行为
2. 表单校验不能完全依赖 UI 组件库的规则，关键业务约束需要显式编码
3. AI 生成的 Vue 组件倾向于使用标准 Element Plus 模板，容易遗漏组件库的边界行为

修复方式对比

维度	纯人工调试	AI 辅助调试
定位耗时	~8 分钟（怀疑校验规则配置 → 尝试加 <code>trigger: 'blur'</code> → 发现无效 → 阅读 Element Plus 源码）	~2 分钟
定位准确度	准确	准确
修复方案	手动加 if 判断	AI 建议在 <code>change</code> 事件中处理 + 手动加 if（双重防线）
最终采用	手动加 if（简单有效）	部分采纳（AI 的双重防线建议过于冗余，未采用）

Bug 8：删除评价后用户平均分重新计算为 null

8

问题现象

管理员删除某用户的唯一一条评价后，该用户的信用页显示 NaN（Not a Number）或 null，前端页面渲染异常。

复现步骤

1. 用户 A 收到唯一一条 5 分评价
2. 用户 A 的平均分为 5.0，信用分显示正常
3. 管理员删除这条评价
4. 再次查询用户 A 的信用信息，creditScore 显示为 NaN

根因分析

后端 ReviewService.java:231-241 中的 recalculateUserAverageScore 方法：

```
private void recalculateUserAverageScore(Long userId) {
    Double newAvg = reviewRepository.getAverageScoreForUser(userId);
    Long newCount = reviewRepository.getScoreCountForUser(userId);

    if (newAvg != null && newCount != null) {
        userRepository.updateUserScore(userId, newAvg, newCount);
    } else {
        userRepository.updateUserScore(userId, 0.0, 0L);
    }
}
```

删除一条评价后，数据库的 getAverageScoreForUser 在 MySQL 中：

- SELECT AVG(score) FROM reviews WHERE reviewed_id = :userId 在没有任何记录时返回 NULL
- SELECT COUNT(*) FROM reviews WHERE reviewed_id = :userId 在没有任何记录时返回 0

逻辑上走到 else 分支：updateUserScore(userId, 0.0, 0L)，应该将平均分置为 0.0。

但问题出在调用链上：deleteReview 方法先执行 reviewRepository.delete(review)，再调用 recalculateUserAverageScore。如果 JPA 的一级缓存或事务提交时机导致 DELETE 在 SELECT AVG 之后才真正执行，getAverageScoreForUser 会读到已删除的评价，返回非 null 值，进入 newAvg != null && newCount == 0 的矛盾状态。

更具体地说：newCount 可能为 0 但 newAvg 不为 null（因为 JPA 查询缓存），导致 newAvg != null && newCount != null 条件成立，但 newCount = 0L，不会进 else。然后 userRepository.updateUserScore(userId, 4.0, 0L) —— 评分数量为 0 但平均分不为 0，前端计算信用分时 creditScore = avgScore * 20 触发除零或 NaN。

根本原因：JPA 的 @Transactional 方法内，先 DELETE 再 SELECT 的查询结果受持久化上下文影响，存在不确定性。

修复方案

方案：在 `deleteReview` 中使用 `@Modifying` 的原生 DELETE 并立即 `flush()`，确保 SELECT 在 DELETE 之后执行。同时增强 `recalculateUserAverageScore` 的防御性：

修改文件： `backend/src/main/java/com/example/backend/service/ReviewService.java`

```
@Transactional
public void deleteReview(Long reviewId, Long adminId) {
    if (!userService.isAdmin(adminId)) {
        throw new RuntimeException("只有管理员可以删除评价");
    }

    Optional<Review> reviewOpt = reviewRepository.findById(reviewId);
    if (reviewOpt.isEmpty()) {
        throw new RuntimeException("评价不存在");
    }

    Review review = reviewOpt.get();
    Long reviewedId = review.getReviewedId();

    reviewRepository.delete(review);
    reviewRepository.flush(); // ← 新增：强制立即执行 DELETE

    recalculateUserAverageScore(reviewedId);
}

private void recalculateUserAverageScore(Long userId) {
    Double newAvg = reviewRepository.getAverageScoreForUser(userId);
    Long newCount = reviewRepository.getScoreCountForUser(userId);

    // 修复：增加防御性判断
    if (newCount == null || newCount == 0) {
        userRepository.updateUserScore(userId, 0.0, 0L);
    } else {
        double safeAvg = (newAvg != null) ? newAvg : 0.0;
        userRepository.updateUserScore(userId, safeAvg, newCount);
    }
}
```

验证结果

- ☒ 删除用户的唯一一条评价后，平均分正确显示 0.0
- ☒ 前端信用页不再出现 NaN
- ☒ 用户有 2 条评价删除 1 条后，平均分正确重新计算
- ☒ 单元测试 `DeleteReviewTests` 全部通过

经验教训

1. JPA `@Transactional` 方法内的 DELETE + SELECT 组合需要注意持久化上下文刷新时机
2. 涉及计算的代码需要防御性编程，不能依赖数据库返回“合理”的值
3. `null/0` 的边界处理在 Java 与 SQL 交汇处特别容易出问题

修复方式对比

维度	纯人工调试	AI 辅助调试
定位耗时	~25 分钟（打印日志 → 排查缓存 → 阅读 JPA flush 机制）	~5 分钟（提供代码和异常日志，AI 直接指出 JPA flush 问题）
定位准确度	准确	准确
修复方案	人工添加 flush	AI 建议 flush + 防御性判断（更完善）
最终采用	—	全部采纳 AI 方案

四、总结

Bug #	类型	模块	严重度	人工耗时	AI 辅助耗时	AI 表现
订单1	字段持久化遗漏	订单模块	P1	—	—	—
订单2	字段持久化遗漏	订单模块	P1	—	—	—
订单3	领域对象未落库	订单模块	P1	—	—	—
消息1	前端事件绑定缺失	消息通知	P1	~10 min	~2 min	优秀
消息2	前端异步加载闪烁	消息通知	P2	—	—	良好
评价1	接口契约不匹配	前后端联调	P0	~15 min	~3 min	优秀
评价2	前端校验漏洞	前端	P1	~8 min	~2 min	良好
评价3	JPA 事务 + 空值处理	后端	P1	~25 min	~5 min	优秀

AI 调试总结

- **AI 的优势：** 在接口契约对比、**框架机制分析（JPA flush）**类 Bug 上表现突出，能快速跨越大量代码定位问题
- **AI 的局限：** 在UI 组件库边界行为（el-rate 事件触发）上需要结合组件文档才能给出准确判断
- **AI 的修复风格：** 修复方案有时会“过度防御”，需人工判断取舍
- **关键成功因素：** 提供完整上下文（报错信息 + 前后端代码 + 浏览器 Network 截图）时 AI 的定位速度显著优于纯人工